

# Informática II

## *Funciones*

Profesor Dr. Paul Bustamante

# Índice

---

- 1. Concepto y Uso**
- 2. Declaración**
- 3. Llamada**
- 4. Sobrecarga de funciones**
- 5. Paso de argumento por valor**
- 6. Paso de argumento por referencia**
- 7. Con arrays**
- 8. Punteros con valor de retorno**
- 9. Funciones recursivas**
- 10. Funciones Inline**

# 1. Concepto y uso

---

**Concepto:** La **función** es una porción de código, un conjunto de sentencias, agrupadas por separado, generalmente enfocadas a realizar una tarea específica. También se suelen denominar subrutinas o **subprogramas**.

**Ventajas:** La división de un programa en unidades más pequeñas o funciones presenta las siguientes ventajas:

- **Modularización:** Es el proceso consistente en dividir un programa en una serie de módulos más pequeños y manejables, llamados funciones. Cada función tiene una misión muy concreta, de modo que nunca tiene un número de líneas excesivo.
- **Ahorro de memoria y tiempo de desarrollo:** En la medida en que una función es utilizada muchas veces, el número total de líneas de código disminuye.
- **Independencia de datos y ocultamiento de información:** Una función es capaz de mantener gran independencia con el resto del programa, manteniendo sus propios datos y definiendo muy claramente la interfaz o comunicación con la función que le ha llamado.

# 1. Concepto y uso

**Definición:** La definición de una función consta de la cabecera de la función y del cuerpo.

Su forma general es:

```
tipo_retorno nombre_funcion (tipo1 argum1, tipo2 argum2, ... ,tipoN argumN)
{
    .....                               //cuerpo de la función
    return (expresion);                  //optativo
}
```

Cada función puede disponer de sus propias variables, declaradas al comienzo de su código. Estas variables son del tipo **auto** y sólo son visibles dentro de la función, es decir, se crean cada vez que se ejecuta la función y permanecen ocultas para el resto del programa.

**La lista de argumentos:** también llamados **argumentos formales**, es una lista de declaraciones de variables, precedidas de su tipo correspondiente y separadas por comas (,).

(Si no se desea pasar ningún valor a la función se puede poner **void** en los paréntesis)

**tipo\_retorno:** valor que devuelve cuando una función es ejecutada (el **valor de retorno**), cuyo tipo debe ser especificado en el encabezamiento de la función.

- La sentencia **return** permite devolver el valor.
- Si no se desea que devuelva nada, el tipo de retorno debe ser **void**.
- Si no se especifica el **tipo\_retorno**, se asume que es **int** por defecto

## 2. Declaración

Toda función debe ser declarada antes de ser utilizada en el programa que realiza la llamada. Esta se hace mediante el **prototipo** de la función.

La forma general del **prototipo** coincide con la primera línea de la definición –el encabezamiento–, con tres pequeñas diferencias:

1. En vez de la lista de argumentos formales o parámetros, basta incluir sólo los tipos de dichos argumentos.
2. El prototipo termina con un carácter “;”.
3. Los valores pueden ser inicializados si se desea.

La declaración de las funciones mediante los prototipos suele hacerse al comienzo del fichero, después de los **#include**.

Declaración

Llamada

Definición

```
#include <iostream>
using namespace std;
double Suma( double, double);
void main(void) {
    double a,b;
    cout << "dar a y b:"; cin >> a >> b;
    double r = Suma(a,b);
    cout << "Suma (a+b): " << r << endl;
}
double Suma( double a, double b){
    double tmp;
    tmp = a+b;
    return tmp;
}
```

## 3. Llamada

La llamada a una función se hace incluyendo su nombre en una expresión o sentencia del programa principal o de otra función. Este nombre debe ir seguido de una lista de argumentos separados por comas y encerrados entre paréntesis. A los argumentos incluidos en la llamada se les llama **argumentos actuales**, y pueden ser no sólo variables y/o constantes, sino también expresiones.

*Declaración  
(con valor por defecto)*

*Llamada con 1 argumento (el  
segundo es constante)*

*Llamada con 2 argumentos*

*Definición de la función*

**Ejemplo 5-1**

```
#include <iostream>
using namespace std;
#include <math.h>

double Potencia(double x, double y=2.0);

void main(void){
    double a,b;
    cout << "dar 2 numeros:";
    cin >> a >> b;
    double pot = Potencia(a);
    cout << "Potencia(a^2): " << pot << endl;
    cout << "Potencia(a^b): " << Potencia(a,b) << endl;
}

double Potencia( double x, double y)
{
    return pow(x,y);
}
```

*Argumentos por omisión  
(siempre al final de la lista de argumentos)*

## 4. Sobrecarga de funciones

Casi siempre es recomendable dar nombres distintos a funciones diferentes, pero

**¿que pasa cuando las tareas a realizar son las mismas pero con *diferentes tipos de datos*?**

En este caso es más conveniente darle a las funciones el mismo nombre, pero con argumentos de distinto tipo, que serán los que permitan al compilador llamar a una u otra función.

*Declaración de  
las dos funciones*

*Llamadas*

*Definición de una  
función*

*Definición de la  
otra función*

**Ejemplo 5-2**

```
#include <iostream>
using namespace std;
void print(int a);
void print(double a);
void main(void)
{
    double a=4.5;
    int b=6;
    print(a);
    print(b);
}
void print(int a)
{
    cout << "Valor entero: " << a << endl;
}
void print(double a)
{
    cout << "Valor real: " << a << endl;
}
```

## 5. Paso de argumentos por valor

Hasta ahora los ejemplos que hemos visto han utilizado este mecanismo. La **llamada por valor** significa que **una copia del argumento se pasa al parámetro de la función**. Esto significa que todo lo que se realice en el código de la función **no va a alterar el valor del argumento usado** para llamar a la función.

**Ejemplo:** al pasar como argumento la variable *r*, se hace una copia en la variable *radio*, la cual es una variable local de la función *AreaCirculo*, por lo tanto los cambios realizados sobre esta variable no afectan a la del programa principal.

**Ejemplo 5-3**

```
#include <iostream>
using namespace std;
double AreaCirculo(double);

void main(void){
    double r=10.0;
    double a;
    a = AreaCirculo(r);
    cout << "El area del circulo de radio " << r ;
    cout << " es: " << a << endl;
}

double AreaCirculo( double radio){
    radio = 3.14 * radio * radio;
    return radio;
}
```



## 6. Paso de argumentos por referencia

Es el mecanismo de **pasar argumentos mediante las direcciones** en lugar de valores. Debe utilizarse siempre y cuando la función deba devolver argumentos modificados.

**Forma 1:** Los **argumentos formales** deben ser declarados como punteros a los **argumentos actuales**, que serán las direcciones.

**Forma 2:** declarando los **argumentos formales** como referencias a los **argumentos actuales**, utilizando el operador "&"

```
#include <iostream>
using namespace std;

void Permutar(double* x, double* y);
void main(void){
    double var1, var2;
    cout << "dar 2 numeros a permutar:";
    cin >> var1 >> var2;
    cout << "Var1: " << var1;
    cout << " Var2:" << var2 << endl;
    Permutar(&var1, &var2);
    cout << "Var1: " << var1;
    cout << " Var2:" << var2 << endl;
}

void Permutar( double* a, double* b){
    double tmp;
    tmp = *a;
    *a = *b;
    *b = tmp;}
```

Puntero  
tipo double

Operador de dirección

No extrae  
ningún valor

Operador de  
indirección

Ejemplo 5-4

```
#include <iostream>
using namespace std;
void Permutar( int &a, int &b);
void main(void)
{
    int x = 25, y = 35;
    cout << "x,y:" << x << ", "<< y
    <<endl;
    Permutar(x,y);
    cout << "x,y:" << x << ", "<< y
    <<endl;
}
void Permutar( int &a, int &b)
{
    int tmp = a;
    a = b;
    b = tmp;
}
```

Variable de referencia (NO ES  
el "Operador de indirección")

Ejemplo 5-5

# Resumiendo

---

Una función es un conjunto de instrucciones C++ que:

1. Es llamado por el programa principal o por otra función
2. Recibe datos a través de una lista de argumentos
3. Realiza una serie de tareas específicas, entre las que pueden estar cálculo y operaciones de lectura/escritura en teclado, pantalla,...
4. Devuelve resultados al programa o función que la ha llamado por medio:
  1. del *valor de retorno*
  2. y de los *argumentos* que hayan sido pasado *por referencia* (punteros o variables referencia)

## 7. Con arrays

En el caso de los **array's**, basta con poner los nombres en la lista de los argumentos. **Un array no se puede pasar como valor, sino sólo como referencia, es decir, se pasa su dirección.**

```
#include <iostream>
using namespace std;
#include <stdlib.h>                                //para rand()

void ImprimeDatos(float*, int);

void main(void){
    int Num = 10;
    float *dat;
    dat = new float[Num];
    for (int i=0;i<Num;i++) dat[i] = (float)rand()/RAND_MAX;
    ImprimeDatos(dat,Num);
    cout << "0:" << dat[0] << endl;
    delete [] dat;
}

void ImprimeDatos( float* datos, int Num ){
    for (int i=0;i<Num;i++) cout << "Dato "<<i<<": " << datos[i] << endl;
    datos[0]=-1.5;    //los cambios aqui afectan al programa principal
}
```

**Ejemplo 5-6**

## 8. Punteros como valor de retorno

El utilizar punteros como valor de retorno permite superar la limitación de devolver un único valor de retorno. Puede devolverse un puntero al primer elemento de un vector o a la dirección base de una matriz.

```
#include <iostream>
using namespace std;
#include <stdlib.h> //para rand()
float* ReservaMemoria(int Num );
void LiberaMemoria(float *vec);
void ImprimeDatos(float* datos, int Num);
void main(void){
    int Num=10; float *dat;
    dat = ReservaMemoria(Num); //llamar a la funcion

    for (int i=0;i<Num;i++) dat[i] = (float)rand()/RAND_MAX; //llenar el vector
    ImprimeDatos(dat,Num);
    LiberaMemoria(dat);
}
void ImprimeDatos( float* datos, int Num ){
    for (int i=0;i<Num;i++) cout << "Dato "<<i<<": " << datos[i] << endl;
}
float* ReservaMemoria( int Num ){
    float *vec;    vec = new float[Num];
    return vec;
}
void LiberaMemoria ( float *vec){
    delete [] vec;
}
```

Ejemplo 5-7

## 9. Funciones recursivas

El C++ es un lenguaje de programación que admite la **recursividad**, esto es, **funciones que pueden llamarse a sí mismas**. Cuando una función es llamada por sí misma, se crea un nuevo juego de parámetros y variables locales, pero el código ejecutable es el mismo. Es muy importante **definir las condiciones** dentro de la función para que la recursividad finalice *y no genere un bucle infinito*.

```
#include <iostream>
using namespace std;
void Recursiva( int );
void main(void)
{
    Recursiva(0); //Llamada
}
void Recursiva( int a)
{
    if (a<10){
        Recursiva(a+1);
        cout << a << " ";
    }
}
```

**Ejemplo 5-8**

La salida en la consola será:

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|

¿Porqué empieza por el 9?

**Ejemplo del cálculo del factorial de un número utilizando la recursividad:**

```
#include <iostream>
using namespace std;
int Factorial( int a);
void main(void)
{
    int num;
    cout << "Factorial de?: ";
    cin >> num;
    cout << "\nEs: " << Factorial(num) << endl;
}
int Factorial( int a)
{
    if (a>1) a *= Factorial(a-1);
    return a;
}
```

**Ejemplo 5-9**

# 10. Funciones Inline

Las funciones **inline** son funciones que no son llamadas sino que son expandidas en línea, en el punto de cada llamada.

Las **ventajas** de estas funciones es que no representan un retardo vinculado con la llamada a la función ni con los mecanismos de vuelta de ésta. Esto significa que las funciones **inline** son ejecutadas de forma mucho más rápida que las normales.

Las **desventajas** de estas funciones es que si son demasiado grandes y son llamadas con demasiada frecuencia, el programa se hace más grande (el tamaño del fichero “.exe” crece).

Para declarar una función **inline** basta con anteponer el especificador **inline** a la definición de la función. Estas deben ser declaradas antes de ser usadas.

Declaración de la función **inline**

Uso de la función **inline**

```
#include <iostream>
inline void Print(int a){
    cout << "Numero: " << a << endl;
}
void main(void)
{
    int a;
    cout << "Dame 1 numero:";
    cin >> a;
    for (int i=0;i<a;i++) Print(i);
}
```

La función **inline** es expandida donde es llamada.

Ejemplo 5-10

# Funciones

| string.h                       |                                                                                                                             |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| int strlen(char *)             | Devuelve la longitud de la cadena                                                                                           |
| strcpy(char *S1, char *S2)     | Copia la cadena S2 en S1                                                                                                    |
| strcat(char *S1, char *S2)     | Agrega la cadena S2 a S1                                                                                                    |
| int strcmp(char *S1, char *s2) | Compara S1 y S2. El resultado es:<br>0: si son iguales las cadenas<br><0: si S1 es menor que S2<br>>0: si S1 es mayor de S2 |

| stdlib.h            |                                    |
|---------------------|------------------------------------|
| int atoi(char *)    | Convertir una cadena en entero     |
| int rand()          | Genera # aleatorio entre 1 y 32767 |
| srand(unsigned int) | Poner punto de inicio para rand()  |
| exit(int)           | Salir del programa                 |

| time.h                        |                                           |
|-------------------------------|-------------------------------------------|
| struct tm                     | Estructura para hora,min,seg,dia,mes,año) |
| time(long *timer)             | Obtener la hora del PC en un long         |
| tm *localtime( long *timer ); | Convertir de long a struct tm             |

| Math.h                 |                                     |
|------------------------|-------------------------------------|
| abs(double x)          | Valor Absoluto                      |
| sqrt(double x)         | Raiz Cuadrada                       |
| pow(double x,double y) | Calcula x^y                         |
| sin(double x)          | Función Seno                        |
| cos(double x)          | Función Coseno                      |
| tan(double x)          | Función Tangente                    |
| exp(double x)          | Función Exponencial                 |
| log(double x)          | Logaritmo Natural                   |
| atof(char *)           | Convierte una cadena en número real |

# Ejemplo

```
#include <iostream>
using namespace std;
#include <time.h>
void ImprimeHora(void);
void Delay(double seg);
void main(void){
    ImprimeHora();
    Delay(2.5);           //retardo de 2.5 seg.
    ImprimeHora();
}
void ImprimeHora(void){
    long t1;
    struct tm *mihora;    //estructura para time
    time(&t1);             //obtener los datos actuales (en long)

    mihora = localtime(&t1); //convierte long a struct tm
    cout << "Fecha: " << mihora->tm_mday << "/" ;
    cout << mihora->tm_mon<<"/"<<mihora->tm_year+1900;
    cout << " Hora: " << mihora->tm_hour << ":" ;
    cout << mihora->tm_min << ":" <<mihora->tm_sec << endl;
}
void Delay(double seg){
    long start;
    long end;
    time(&start);
    end = start;
    while( difftime(end,start) < seg){
        time(&end);
    }
}
```